VINUNIVERSITY

DPO, ORPO & Alignment — Từ SFT đến Preference Learning

AICB-P2T3 · Ngày 22 · Chương 5 — Fine-tuning & An Toàn

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 5

HÃY SUY NGHĨ...

“RLHF tốn kém — DPO làm được điều tương tự mà không cần reward model?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1. Tại sao SFT chưa đủ?
2. RLHF — Bức tranh toàn cảnh
3. DPO — Direct Preference Optimization
4. ORPO, SimPO & Alternatives
5. RL comeback — GRPO & RLVR
6. Preference Data & Implementation
7. Constitutional AI & Red-teaming

Tại sao SFT chưa đủ?

01

SFT dạy “nói gì” — nhưng ai dạy model “nói như thế nào”?

SFT dạy format — Alignment dạy hành vi



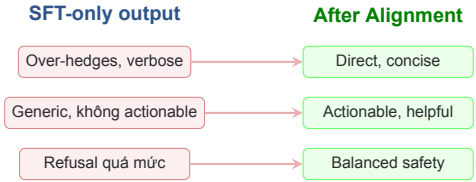
Post-training pipeline hiện đại:

1. **SFT**: Dạy model format câu trả lời (instruction following)
2. **Alignment**: Dạy model phân biệt tốt/xấu (helpful, harmless, honest)

Practical framing

SFT dạy model "nói gì".
DPO/ORPO dạy model "nói **như thế nào**" — concise, helpful, an toàn.

SFT-only: vấn đề gì xảy ra?

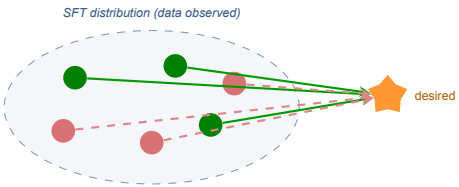


> **Alignment** — Quá trình dạy model phân biệt câu trả lời **tốt** vs **xấu** bằng preference data — không phải dạy thêm kiến thức mới.

Kết quả nổi bật:
InstructGPT **1.3B** với RLHF
$>$ GPT-3 **175B** không RLHF

$\Rightarrow$ *Alignment beats scale* (Ouyang et al. 2022)

Vì sao preference data mạnh hơn demonstration data?



SFT chỉ thấy "good"→ không biết good hơn bad bao nhiêu

SFT (demonstration): "Bắt chước câu này."
⇒ Học **distribution** của data.

Preference (DPO/RLHF): "Câu A *tốt hơn* câu B."
⇒ Học **margin** giữa good vs bad.

Information signal

Một preference pair (y_w, y_l) chứa thông tin về **cái gì KHÔNG nên nói** — điều mà SFT data không bao giờ biểu lộ trực tiếp.

02

RLHF — Bức tranh toàn cảnh

Hiểu RLHF để thấy tại sao DPO là bước cải tiến

RLHF Pipeline — InstructGPT 3-Stage Architecture



Lưu ý: RLHF pipeline phức tạp: **3 models, 3 stages**, nhiều hyperparams ⇒ chỉ frontier labs (OpenAI, Anthropic) có đủ resources để dùng.

Analogy

RLHF = **thuê giám khảo chấm điểm**, rồi dùng điểm đó dạy lại model chính. Tốn kém gấp đôi.

RLHF: Kết quả tốt nhưng chi phí cao

VINUNIVERSITY

3
Models cần
train đồng thời

Cao
PPO instability
reward hacking

$$$$
Chi phí tổng
(infra + annotators)

Câu hỏi then chốt

Nếu ta có thể **bỏ Reward Model** và train trực tiếp trên preference data thì sao? $\Rightarrow$ Đó chính là ý tưởng của **DPO**.

DPO — Direct Preference Optimization

03

Bỏ Reward Model, train trực tiếp trên cặp tốt/xấu

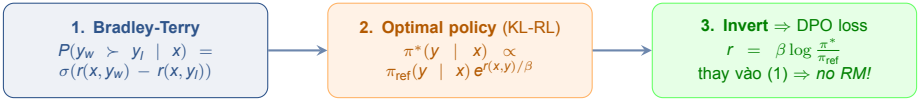
DPO (Rafailov et al. 2023)

Key insight: optimal RLHF policy có **closed-form solution**. Vì vậy ta có thể train trực tiếp trên preference data mà **không cần Reward Model**.

Analogy: RLHF = thuê giám khảo chấm điểm rồi dùng điểm đó dạy lại. **DPO = cho model xem trực tiếp cặp tốt/xấu, tự học phân biệt** — bỏ qua giám khảo trung gian.

DPO loss: binary cross-entropy trên log-ratio chosen vs rejected probabilities.

DPO derivation — 3 bước từ Bradley-Terry đến loss



Magic trick

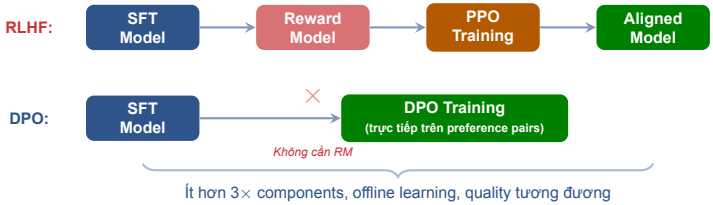
"Closed-form optimal policy" (bước 2) là chìa khóa. Nếu không có nó, ta không thể bỏ Reward Model. DPO chỉ áp dụng được cho objective dạng **KL-regularized RL** — không phải mọi RL setup.

DPO loss (final form):

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}\right)$$

Forward pass qua π_θ và π_{ref} trên (prompt, y_w, y_l). Không rollout, không PPO clipping. *(Rafailov et al. 2023)*

RLHF vs DPO — So sánh Pipeline

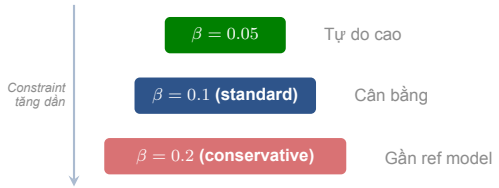


DPO advantages

Offline learning (stable)
Ít components: chỉ 1 model + ref
Quality tương đương RLHF trên benchmarks

Lưu ý: Known issues: **length bias** (model học viết dài hơn thay vì tốt hơn), **ref model dependency**, over-fitting trên small datasets.

DPO Hyperparameter: β — Kiểm soát mức độ alignment



β (KL penalty) — β cao $\Rightarrow$ model bị giữ gần ref model hơn. β thấp $\Rightarrow$ model tự do "rời xa" ref model để optimize preference.

- Bắt đầu $\beta = 0.1$ (default)
- Nếu model quá conservative $\Rightarrow$ giảm xuống 0.05
- Nếu model "quên" kiến thức $\Rightarrow$ tăng lên 0.2

IPO: variant fix length bias — drop-in replacement cho DPO.

DPO failure modes — khi training "thành công" nhưng model tệ hơn

Lưu ý: **Likelihood displacement** (Razin 2024) — prob của *chosen* **giảm** khi train; khi chosen ≈ rejected, mass dồn sang token **ngược nghĩa** ⇒ unalignment.

Lưu ý: **Length hacking** — DPO thưởng response dài (nhiều log-prob mass). Học "viết dài" thay vì "viết tốt".

Lưu ý: **Mode collapse / sycophancy** — mọi câu mở đầu "Great question!" — over-fit preference style.

Triệu chứng trên TRL logs:

- `rewards/chosen` **giảm** ⇒ likelihood displacement
- `rewards/margins` âm/co ⇒ optimization conflict
- Length tăng $>30\%$ ⇒ length hacking

Mitigations

Filter pairs có gap quá nhỏ; dùng **SimPO**/**IPO**; stop sớm khi `rewards/chosen` đảo chiều.

ORPO, SimPO & Alternatives

04

Single-stage alignment — bỏ cả bước SFT riêng biệt

Alignment timeline 2022 → 2026 — từ PPO đến RLVR



Cốt truyện

2022: RLHF nặng, 3 models, PPO unstable.
2023: DPO — toán bỏ được RM.
2024: method nở rộ, mỗi method bỏ thêm *một thứ*
(ref model, SFT stage, length bias).
2025: RL trở lại với GRPO/RLVR — nhưng *không reward model*.

Lưu ý: Tooling đổi liên tục, nhưng câu hỏi không đổi: "**Ai dạy model phân biệt good vs bad?**" Khi câu trả lời là người → RLHF/DPO. Khi là code/regex → RLVR. Khi là chính model → Self-Rewarding / CAI.

Refs: Ouyang 2022, Rafailov 2023, Hong/Meng/Ethayarajh 2024, Tulu 3 2024, DeepSeek-R1 2025.

Method matrix — chọn alignment method nào?

Method	RM?	Ref?	Stages	Khi nào dùng?
RLHF (PPO)	Có	Có	3	Frontier labs, max quality
DPO	Không	Có	2	Go-to production alignment
IPO	Không	Có	2	DPO over-fit deterministic prefs
SimPO	Không	Không	2	Length-norm, less VRAM
ORPO	Không	Không	1	Base → aligned 1 stage
KTO	Không	Có	2	Chỉ có thumbs up/down
GRPO	Không	Có	1 RL	Reasoning + RLVR

Quick rules

Có SFT + pref pairs ⇒ **DPO**. No SFT ⇒ **ORPO**. Chỉ **+1**/**−1** ⇒ **KTO**. Math/code ⇒ **GRPO+RLVR**.

Lưu ý: DPO vẫn là **baseline tốt nhất** 2025-2026. Chuyển method khác chỉ khi có lý do cụ thể (VRAM, no SFT, reasoning).

SimPO vs KTO — hai hướng đi sau DPO

SimPO (Meng et al. 2024)

Reference-free — không cần π_{ref}.
Implicit reward = **average log-prob** của response (length-normalized).
+ Target margin γ trên Bradley-Terry.

Kết quả: +6.4 trên AlpacaEval 2, +7.5 trên Arena-Hard so với DPO (§8.3). Top SimPO model (Gemma-2-9B-it) đạt 72.4% LC win-rate.

Khi nào dùng: VRAM hạn chế, length bias là vấn đề lớn.

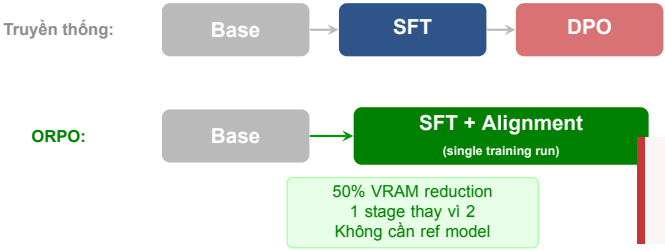
KTO (Ethayarajh et al. 2024)

Single-signal — không cần preference *pairs*.
Mỗi example chỉ cần label **good/bad** (**+1**/**−1**).
Loss dựa trên *prospect theory* (Kahneman-Tversky): mô hình loss aversion.

Lợi thế thực tế: dữ liệu thumbs-up/down từ production logs dễ thu thập hơn nhiều so với ranked pairs.

Khi nào dùng: có user feedback ratings từ production, không có annotators chuyên dụng.

ORPO — Single-Stage: Base → Aligned



- ORPO kết hợp SFT loss + preference loss trong cùng 1 objective
- Best case: **base model → aligned model in one step**
- Skip SFT stage hoàn toàn

Lưu ý: ORPO trade-off: đơn giản hơn nhưng ít mature hơn DPO. Dùng khi muốn train từ base model và VRAM hạn chế.

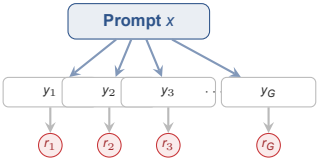
GRPO (DeepSeek-R1): next frontier cho reasoning alignment — thay thế PPO bằng group relative policy, không cần RM.

05

RL comeback — GRPO & RLVR

Năm 2025: RL trở lại cho reasoning, nhưng **không cần reward model**

GRPO — PPO không có value model



GRPO — Group Relative Policy Optimization. Lấy trung bình reward của **nhóm** G **samples** cho cùng prompt làm baseline — thay thế cho value model trong PPO.

- PPO clipping ratio như cũ
- **Bỏ value head** $\Rightarrow$ 50% memory savings
- Dùng cho DeepSeekMath, sau đó DeepSeek-R1 (Jan 2025)

Refs: Shao et al. 2024 (DeepSeekMath); DeepSeek-AI 2025 (R1).

RLVR & DeepSeek-R1 — reward thay bằng *regex*

RLVR — Reinforcement Learning from **Verifiable** Rewards. Reward = **kiểm tra programmatic** (math match, unit test, regex). Không có "judge" để hack.

Khi nào RLVR work:

- Math: ground truth trong dataset
- Code: chạy unit tests
- Format: regex / JSON schema
- Tool use: success/error from tool

Lưu ý: RLVR *không* thay preference learning cho subjective tasks. **Bổ sung**, không thay thế.

DeepSeek-R1-Zero (Jan 2025):

- Train **base model** (không SFT cold-start)
- Reward = accuracy + format (rule-based)
- Emergent "Aha!" self-reflection

R1 (full): thêm SFT cold-start nhỏ → readability tốt hơn, math reasoning tương đương.

Hook

2023: "RLHF tốn kém, dùng DPO." 2025: "RL trở lại cho reasoning — *nhưng không reward model*."

Khi nào reasoning task cần RL chứ không phải DPO?

Task type	Method khuyến nghị
Helpfulness, style, tone	DPO / SimPO
Safety, harmlessness	DPO + CAI
Math word problems	GRPO + RLVR (answer match)
Code generation	GRPO + RLVR (unit tests)
Long-form reasoning	SFT cold-start + GRPO
Tool calling correctness	GRPO + RLVR (tool result)
Multi-turn dialogue	DPO (preference pairs)

Quy tắc đơn giản

Có **ground truth** có thể check programmatic? $\rightarrow$ **RLVR**.
Chỉ có **judgment** của con người? $\rightarrow$ **DPO**.
Cả hai? $\rightarrow$ Stack: SFT $\rightarrow$ DPO $\rightarrow$ RLVR (Tulu 3 recipe).

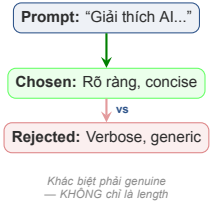
Lưu ý: GRPO **không miễn phí**: cần generation rollouts ($G \approx 8$–64 samples per prompt) $\Rightarrow$ 3-4$\times$ thời gian so với DPO. Chỉ dùng khi reasoning task thực sự cần.

Preference Data & Implementation

06

Chuẩn bị dữ liệu preference và chạy DPO với TRL

Preference Dataset — prompt + chosen + rejected



Nguồn dữ liệu:

- **Human annotation**: pairwise comparison UI → JSONL export
- **Synthetic**: GPT-4 judge score, rank, tạo pairs
- Tools: Argilla, Label Studio, Prodigy

Quy mô tham khảo

60K pairs cho good alignment. Open-source: UltraFeedback (64k), Anthropic HH (160k), OpenHermes (1M).

```python
from trl import DPOTrainer, DPOConfig
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
    "path/to/sft-model")
ref_model = AutoModelForCausalLM.from_pretrained(
    "path/to/sft-model")  # frozen
config = DPOConfig(
    beta=0.1, learning_rate=5e-7,
    max_length=1024, max_prompt_length=512,
)
trainer = DPOTrainer(
    model=model, ref_model=ref_model,
    args=config, train_dataset=pref_data,
)
trainer.train()
```

Monitoring healthy training:

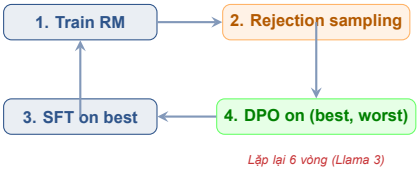
- **Chosen rewards**: phải **tăng** qua epochs
- **Rejected rewards**: phải **giảm** qua epochs
- Gap giữa 2 loại nên **widen** dần

Alternatives

SimPO: không cần `ref_model` ⇒ simpler, less VRAM.
ORPOTrainer: single-stage, không cần SFT trước.

Lưu ý: Tokenization: verify `max_prompt_length + max_length` fit context window. Truncation giảm quality.

Iterative DPO — vì sao one-shot offline không đủ



Llama 3 recipe (Meta 2024):

- Train RM trên human preference + edited responses
- Sinh 10–30 generations / prompt → RM chọn best
- SFT trên best, DPO trên (best vs worst)
- **Lặp 6 vòng** với data mới mỗi vòng

Tulu 3 (AI2, Nov 2024)

Open recipe: **SFT** → **DPO** → **RLVR**. RLVR thêm +1.7 MATH / +3.3 GSM8K / +1.3 IFEval trên DPO checkpoint — transfer cả ra BBH, DROP, AlpacaEval. *(Định nghĩa benchmark: §8.2–8.3.)*

Lưu ý: **Lesson:** one-shot offline DPO underperforms. Mọi 2024-2025 stack đều iterative.

Reward models 2025 — chưa biến mất, chỉ đổi vai trò

VINUNIVERSITY

Trong DPO world, RM dùng làm gì?

- **Data filter**: loại bỏ pairs gap quá nhỏ (chosen $\approx$ rejected)
- **Rejection sampling**: chọn best-of-N generations cho SFT next round
- **Best-of-N decoding**: tại inference, sinh N câu, pick best
- **LLM-as-judge backbone**: thay GPT-4 cho eval rẻ hơn

Lưu ý: Đừng nhầm: "Không cần RM cho DPO loss" *không* có nghĩa "Không cần RM ở đâu cả". RM vẫn ở khắp pipeline — chỉ không trực tiếp trong gradient update.

Top open RMs (2025):

- **Skywork-Reward-V2** (Jul 2025): 8 models 0.6B–8B, 26M pairs, SOTA trên 7 benchmarks.
- **HelpSteer2** (NVIDIA): 10K pairs quality cực cao — former SOTA.
- **RewardBench v2**: standard benchmark (§8.4).

Default

`Skywork-Reward-V2-Llama-3.1-8B`. Dùng làm filter cho rejection sampling + best-of-N.

Vietnamese LLMs — ai đã làm DPO, ai chưa?

VINUNIVERSITY

Model	Base	SFT	DPO?	Ghi chú
VinaLLaMA-7B-Chat	LLaMA-2-7B	✓	×	VN instruction set; chỉ SFT (paper 2023)
PhoGPT-7B5-Instruct	PhoGPT (VN scratch)	✓	×	70K instructions + 290K chats; no preference stage
PhoGPT-4B-Chat	PhoGPT-4B	✓	×	Smaller chat variant
Vistral-7B-Chat	Mistral-7B	✓	×	Community SFT
SeaLLM-v2/v2.5/v3	Llama-2 / Qwen	✓	✓	DPO with self-generated preference data (DAMO)
Sailor / Sailor2	Qwen 1.5/2.5	✓	✓	DPO + variants (SimPO, LN-DPO, LR-DPO) (Sea AI Lab)

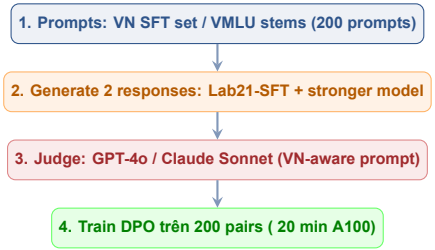
Pattern

VN-first (VinaLLaMA, PhoGPT, Vistral) dừng ở SFT. *SEA-regional* (SeaLLM, Sailor) chạy tới DPO. ⇒ Gap cho VN-first DPO-aligned model.

Lưu ý: Lab 22 có thể là DPO-aligned VN model open-source đầu tiên end-to-end của khóa — publishable.

Refs: VinaLLaMA 2312.11011 · PhoGPT 2311.02945 · SeaLLM 2312.00738 · Sailor (EMNLP 2024) · Sailor2 2502.12982.

Xây dựng preference data tiếng Việt — 4 bước thực tế



Nguồn dữ liệu VN sẵn có:

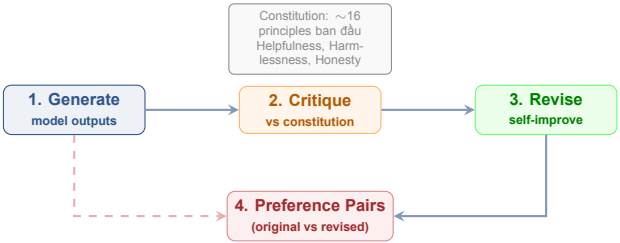
- **UltraFeedback-vi**: Sailor dịch bằng NLLB-3.3B — "đủ cho DPO" nhưng không native.
- **SeaLLM self-generated**: prompt SeaLLM-SFT, GPT-4 judge.
- **VMLU / ViGLUE**: có ground truth → dùng làm **RLVR rewards** cho academic knowledge (§8.5).

> **Cơ hội VN**
> Chưa có *native* large-scale preference dataset — lab artifact đáng publish.

07

Constitutional AI & Red-teaming

Tạo preference data tự động và kiểm tra an toàn



CAI — Model critique own outputs vs constitution → self-revise → dùng cặp (original, revised) làm preference data.

Ưu điểm:

- Tạo preference data **không cần human annotators**
- Scalable, consistent
- Dùng cho DPO/ORPO training

RLAIF — Reinforcement Learning from AI Feedback (Lee et al. 2023). Thay human annotators bằng **LLM-as-judge**. Empirically: AI feedback $\approx$ human feedback ở scale, 10$\times$ cheaper.

Self-Rewarding LM — (Yuan et al., Meta 2024) Model là chính judge của mình. Lặp: generate $\rightarrow$ self-judge $\rightarrow$ form pairs $\rightarrow$ DPO. Sau 3 vòng, Llama-2-70B beats Claude 2 / Gemini Pro / GPT-4-0613 trên AlpacaEval 2.

Collective Constitutional AI (Anthropic 2024):

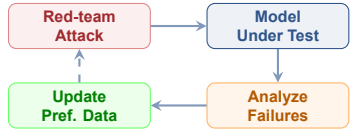
- Mời **public input** vào constitution document
- Polis-style consensus elicitation
- Constitution không còn là 16 nguyên tắc do Anthropic viết — mà là tinh thần cộng đồng

Pattern chung

Ai sẽ làm judge? Human (RLHF) $\rightarrow$ AI (RLAIF) $\rightarrow$ Self (Self-Reward) $\rightarrow$ Community (Coll. CAI). Rẻ hơn, scale hơn — nhưng risk bias tăng theo.

Red-teaming — Kiểm tra an toàn trước deploy

- Probe model cho **harmful outputs**
- Findings feed back vào preference dataset
- **Domain-specific**: medical misinformation, legal liability



Continuous improvement loop

Automated Tools

Garak: vulnerability scanner
PyRIT (Microsoft): adversarial testing
$\Rightarrow$ Scalable, repeatable

Đánh giá Alignment — LLM Benchmarks

Static suites · LLM-as-Judge · Reward Bench · Vietnamese land-scape

Tại sao đánh giá alignment khó?

VINUNIVERSITY

Vấn đề cốt lõi

Aligned response là **open-ended** — không có 1 ground-truth duy nhất:

"Giúp tôi viết email xin nghỉ phép" → vô số câu trả lời đều đúng.

Khác hẳn classification (1 nhãn đúng) hay translation (BLEU vs reference).

Proxy vs Target

Cái ta *đo* (helpfulness 1-5) là **proxy** — không phải target thật.

Target thật: user retention, task completion, brand trust, không gây hại.

Mọi metric chỉ là 1 lát cắt ⇒ **cần nhiều metrics + human feedback**.

Lưu ý: **3 nhóm benchmark** sẽ xuất hiện trong các slide tiếp theo: **Static suites** (MMLU/GSM8K — đo capability) · **Judge-based suites** (MT-Bench/AlpacaEval — đo response quality) · **Reward Model suites** (RewardBench — đo chính các judges). Không 1 cái nào đủ một mình.

Static benchmarks — đo capability của aligned model

VINUNIVERSITY

Benchmark	Đo gì	Format	Score	Vì sao quan trọng cho aligned model
MMLU	Kiến thức nền 57 subjects	MCQ 4-choice	0–100%	Có quên kiến thức sau alignment?
GSM8K	Math grade-school	Gen + match ####	0–100%	Chat-tuning có giảm reasoning? (alignment t
MATH	Math thi olympic	Gen + LaTeX match	0–100%	Như GSM8K, harder ($\sim$50 chỉ top model)
IFEval	Theo lệnh format	Gen + programmatic	0–100%	Aligned model có nghe "trả lời $\leq$3 câu"?
HumanEval	Code generation	Run unit tests	0–100%	Code aligned có chạy được không?
BBH	Mixed reasoning 23 tasks	MCQ + gen	0–100%	Stress-test đa-nhiệm
TruthfulQA	Truthfulness vs myths	MCQ + gen	0–100%	Có hallucinate "common belief" sai?

Tulu 3 reference

Sau DPO + RLVR (xem §9.2b): **+1.7 MATH · +3.3 GSM8K · +1.3 IFEval** — nhưng MMLU thường *flat* hoặc *giảm nhẹ* sau chat-alignment.

Tất cả đều programmatic scoring; chạy offline trên 1 GPU.

Judge-based benchmarks — đo response quality

Benchmark	# prompts	Format	Judge	Tradeoff
MT-Bench	80 multi-turn	Score 1–10	GPT-4	Tiny set; judge có position bias
AlpacaEval 2 LC	805 single-turn	Win-rate vs gpt-4-1106	GPT-4	Length-controlled (Dubois 2024) sửa length-bias
Arena-Hard	500 hard prompts	Win-rate vs gpt-4-0314	GPT-4	Khó cho weak models; thẩm chí gpt-3.5 cũng $<30\%$
Chatbot Arena	live, >3M votes	ELO ranking	Người dùng thật	Ground-truth nhất, nhưng đắt + chậm

Cross-judge tip

Chạy cùng 1 prompt qua **2 judges khác nhau** (gpt-4o-mini + claude-haiku-4-5); disagreement = signal cần xem tay.

Lưu ý: **Length bias** (judges thiên vị câu dài) là failure mode lớn nhất pre-2024. AlpacaEval 2 *LC* fixes nó; MT-Bench thì chưa.

Reward Models = meta-judge — và vấn đề recursion

VINUNIVERSITY

Tại sao cần benchmark cho RM/-judges?

Judge cũng là 1 model ⇒ có bias.

RewardBench v2 (Lambert et al. 2024) test RM trên 4 categories: chat · safety · reasoning · code.

RM-Bench (Skywork 2024) thêm hard pairs phân biệt model gần nhau.

Tulu 3 dùng **Skywork-Reward-Gemma** làm *filter* cho preference data trước khi train DPO.

Lưu ý: Vấn đề recursion: judge cần judge, RM cần benchmark, benchmark cần evaluation — không có "ground truth" tuyệt đối ngoài **con người + thời gian + production data**. Constitutional AI (§7) là 1 cách *thoát recursion*: dùng **explicit rules** làm anchor thay vì preferences.

Trong lab

NB4 = judge-based, **NB6 = static** — 2 lăng kính khác nhau, kết quả khác nhau là bình thường.

Vietnamese benchmark landscape — gì có, gì thiếu

Benchmark	Đo gì	Status	Note
VMLU	10K MCQ, 58 VN subjects	Active 2024+	Pham et al. — "lò luyện thi" THPT + ĐH
ViGLUE	NLU 5 tasks (NER, sentiment, NLI, QA)	2023 stable	Older, vẫn dùng được cho NLU baseline
ViMMLU	MMLU dịch sang Việt	2023	NLLB-MT quality concerns; OK cho rough c
VLSP shared tasks	Various NLP tasks	Annual	Academic; dataset rotates yearly
VN AlpacaEval	*Win-rate VN*	**GAP**	*Chưa tồn tại! Bonus B opportunity*

Lưu ý: **Native VN judge-based benchmark chưa tồn tại.** Sailor2 dùng UltraFeedback-vi (translated). Cơ hội: ai trong cohort này build 1 *native* VN AlpacaEval-style set 200–500 prompts $\Rightarrow$ **publishable đầu tiên** (xem `BONUS-CHALLENGE.md` provocation #1).

Demo & Thực hành

09

DPO training thực tế + đo improvement bằng GPT-4 Judge

DPO Training + LLM-as-Judge Comparison

1. Lấy SFT checkpoint từ Lab 21, apply DPO với 2k UltraFeedback pairs, 1 epoch
2. Before DPO: over-hedges, generic, verbose. After DPO: direct, concise, actionable
3. GPT-4 judge: helpfulness từ **3.2** $\rightarrow$ **4.1** out of 5
4. So sánh: chosen rewards tăng, rejected rewards giảm — healthy training signals

DPO Improvement — Before vs After

3.2 → 4.1
Helpfulness
(GPT-4 judge)

−40%
Response length
(less verbose)

1 epoch
Training time
(∼30 min A100)

Tại sao hiệu quả?

DPO dạy model phân biệt **trực tiếp** giữa good vs bad response — model học preference signal rất nhanh, chỉ cần 1–2 epochs.

Tulu 3 (AI2 2024) — SFT vs +DPO vs +RLVR thực tế

+1.7
MATH
sau RLVR (vs DPO)

+3.3
GSM8K
sau RLVR

+1.3
IFEval
sau RLVR

Đọc bảng số

Tulu 3 publish toàn bộ recipe + data + code. Mỗi stage thêm *vài points*, không phải đột phá — nhưng **cộng dồn**: SFT → DPO → RLVR đủ để Tulu 3-405B vượt Llama 3.1-Instruct-405B và DeepSeek-V3 trên AI2 eval suite. Đây là benchmark thực tế của "modern alignment stack".

Mục tiêu: DPO alignment trên SFT checkpoint + deploy aligned model

Deliverable: Aligned model deployed: merge adapter, quantize, serve với vLLM. Report: SFT-only vs SFT+DPO comparison.

Thời gian: 2 giờ

Lab 22 — Các bước thực hành

1. **Prepare preference dataset**: human-ranked response pairs (hoặc synthetic via GPT-4 judge), format prompt/chosen/rejected
2. **Train DPO adapter**: trên SFT checkpoint từ Lab 21 dùng TRL DPOTrainer. Monitor chosen/rejected rewards
3. **Compare**: SFT-only vs SFT+DPO trên safety và helpfulness metrics. Dùng GPT-4 judge cho evaluation
4. **Deploy**: merge adapter, quantize GGUF, serve với vLLM. Measure latency overhead

Deliverable

GitHub repo + comparison report: helpfulness score trước/sau DPO, safety metrics, example outputs

Lab 22 — 2 bonus tracks (optional, +1 giờ mỗi track)

VINUNIVERSITY

Bonus A

DPO vs ORPO head-to-head

1. Cùng base model, cùng preference data
2. Train 1 adapter DPOTrainer, 1 ORPOTrainer
3. So sánh: judge win-rate + time + VRAM
4. Plot: chosen/rejected reward trajectories

Outcome: hiểu trade-off "đơn giản vs mature".

Bonus B

Vietnamese preference data

1. 200 prompts từ VN SFT set / VMLU stems
2. 2 responses: Lab21-SFT + stronger (Gemini Flash / Claude Haiku)
3. Judge: GPT-4o / Claude Sonnet (VN-aware prompt)
4. Train DPO 200 pairs, so sánh native vs UltraFeedback-vi

Outcome: dataset + adapter publish-able — fill VN gap.

Bức tranh toàn cảnh — Full training flow

Alignment chỉ là một mảnh ghép — xem nó nằm ở đâu trong vòng đời của LLM

Pre-training → Mid-training → Post-training

Pre-training	Mid-training	Post-training
Trillions of tokens, next-token prediction	Continued pretraining, domain adapt, long-context extension	SFT → DPO → RLVR; merge, distill, optimize
Cost: rất cao (M USD) Frontier labs only	Cost: cao Domain teams	Cost: vừa phải *Most ML teams live here*

Insight

99% công việc của ML team trong industry rơi vào **post-training** (+ một ít mid-training). Pre-training là sân chơi của 5-10 lab toàn cầu (OpenAI, Anthropic, Google, Meta, Mistral, Alibaba, DeepSeek…).

Lưu ý: Khi nào cần pre-training? Hầu như **không bao giờ** cho startup/enterprise. Default: bắt đầu từ Llama / Qwen / Mistral base + post-train. Pre-training chỉ khi domain thực sự ngoài distribution (vd: protein sequences).

Domain adaptation — DAPT vs TAPT vs LoRA fine-tune

Method	Data	Compute	Khi nào dùng?
DAPT (continued pre-training)	Raw domain text (10B+ tokens)	cao	Domain hoàn toàn ngoài distribution (medical, legal, code)
TAPT (task-adaptive PT)	Unlabeled task corpus	vừa	Có sẵn task corpus, label chưa có
SFT / LoRA fine-tune	Labeled instruction pairs	thấp	Đã có format và task rõ ràng
DPO / preference align	Preference pairs	thấp	Behavior alignment, không phải domain knowledge

Sequential recipe

Khi cần cả 2: **DAPT** → **TAPT** → **SFT** → **DPO**. Mỗi bước thêm vài points. (Gururangan et al. 2020 “Don’t Stop Pretraining”.)

Lưu ý: **Catastrophic forgetting** là rủi ro lớn nhất của DAPT/TAPT. **Replay 50/50**: trộn 50% domain + 50% general (Pile, RedPajama).

Synthetic data generation — 4 thế hệ kỹ thuật

1. **Self-Instruct** (Wang 2022): seed prompts, LLM tự sinh thêm. Quality thấp, scale dễ. (Alpaca lineage.)
2. **Evol-Instruct** (WizardLM 2023): lặp prompt rewriting để tăng độ khó (in-depth, in-breadth, deepening).
3. **Persona-driven** (Persona Hub 2024): 1B personas → diverse perspectives trên cùng prompt.
4. **Verified synthesis** (2024+): LLM sinh response, sau đó *kiểm tra programmatic* (chạy code, regex check, tool call result) trước khi đưa vào dataset — vd. WizardCoder, OpenMathInstruct, Tulu-3 RLVR data.

Phi recipe

Phi-3/Phi-4 đặt cược vào synthetic "textbook-like": structured, step-by-step. **Phi-4 vượt teacher GPT-4 trên STEM** — distillation không còn là trần.

Lưu ý: **Pitfalls:** mode collapse khi self-generate; license risk khi dùng GPT-4/Claude làm teacher; quality decay sau 2–3 thế hệ synthetic.

Distillation lineage — Alpaca → Vicuna → Orca → Phi

Alpaca (2023)	Vicuna/WizardLM (2023)	Orca (2023-24)	Phi-3/Phi-4 (2024-25)
52K Self-Instruct, GPT-3.5 teacher, imitate response	ShareGPT logs, Evol-Instruct, scale up	GPT-4 teacher, copy *reasoning traces*, step-by-step CoT	Synthetic “textbooks”, curriculum-driven, surpasses teacher on STEM

Tiến hóa của distillation

Gen 1: copy outputs. Gen 2: copy at scale. Gen 3: copy reasoning. Gen 4: synthesize structured curriculum → student *vượt* teacher.

Lưu ý: **Legal:** OpenAI/Anthropic/Google ToS cấm dùng output để train competing model. Lab/research thường OK; commercial cần legal review. Open alternatives: Llama-3 70B, Qwen-2.5-72B, Mixtral-8x22B teachers — license cho phép commercial distillation.

Hardware optimization — compute vs memory trade-off

FlashAttention (v1/v2/v3) — Attention exact, nhưng tile-by-tile, giữ data trong SRAM thay vì HBM. **2–4× speedup**, **O(N)** memory thay vì $O(N^2)$. v3 (Hopper H100) thêm async + FP8.

Gradient checkpointing — Không lưu activations trong forward; recompute trong backward. **Trade memory cho compute**: 30–50% memory savings, 30% slower. Bắt buộc cho long-context FT.

Combine cả hai:

- FlashAttn ⇒ tăng max sequence length
- Grad ckpt ⇒ tăng max batch size
- BF16 mixed precision ⇒ 50% memory
- NF4 4-bit ⇒ thêm 50%

Order khi OOM

1. Mixed precision → 2. Grad ckpt → 3. FlashAttn 2 → 4. 4-bit (QLoRA) → 5. Gradient accumulation → 6. ZeRO/FSDP (multi-GPU).

Distributed training — ZeRO Stages & FSDP

VINUNIVERSITY

Stage	Shard	Savings	Khi nào dùng?
Stage 0 (DDP)	Replicate everything	1×	1 GPU đủ memory
ZeRO-1	+ Optimizer state	4×	2–4 GPUs cùng node
ZeRO-2	+ Gradients	8×	4–8 GPUs cùng node (default)
ZeRO-3 / FSDP	+ Parameters	N×	Model > single-GPU memory

Quick guide

1 GPU: không cần ZeRO — dùng QLoRA + grad ckpt. **2–8 GPUs:** ZeRO-2 (DeepSpeed) hoặc FSDP. **>8 GPUs / multi-node:** FSDP + selective ckpt; Stage 3 khi cần.

Lưu ý: **FSDP gotcha:** dùng *non-reentrant* activation checkpointing với FSDP — legacy reentrant variant xung đột với FSDP state sync. Selective activation ckpt (PyTorch 2024+) thêm 10% throughput.

Modern PEFT variants — DoRA, rsLoRA, PiSSA

DoRA — Weight-Decomposed LoRA (Liu 2024). Tách update = **magnitude** + **direction**. Tốt hơn LoRA ở rank thấp.

rsLoRA — Rank-Stabilized (Kalajdzievski 2023). Scaling $\alpha/\sqrt{r}$ thay vì $\alpha/r \Rightarrow$ stable gradient ở rank cao ($r = 64$–128).

PiSSA — Principal SV Adaptation (NeurIPS 2024). Init LoRA bằng top-SVD của W_0 thay vì random. +5.16% trên GSM8K (Mistral-7B).

Decision guide

Default: LoRA $r = 8$–16, $\alpha = 16$–32. **Low rank** ($r \leq 4$): DoRA. **High rank** ($r \geq 32$): rsLoRA. **Convergence nhanh**: PiSSA. **VRAM cực hạn**: QLoRA+NF4.

Lưu ý: **2025:** với LR tuning đúng, mọi variant peak tương đương vanilla LoRA (Liu 2025). Tune LR trước, đừng over-engineer.

Hỗ trợ trong HuggingFace PEFT $\geq$0.10.

Model merging — combine fine-tunes mà không retrain

- **SLERP**: spherical linear interpolation, merge *2 models* dọc đường cầu giữ angle. Đơn giản, hiệu quả khi 2 models share base.
- **Task Arithmetic**: $\theta_{\text{merged}} = \theta_{\text{base}} + \sum_i \lambda_i(\theta_i - \theta_{\text{base}})$. Cộng/trừ "task vectors".
- **TIES** (Trim, Elect Sign, Merge): trim 80% delta nhỏ nhất, vote sign chung $\rightarrow$ giảm interference.
- **DARE**: drop 90–99% delta ngẫu nhiên rồi rescale. Surprising: vẫn giữ performance.
- **DARE-TIES**: kết hợp 2 method trên cho multi-model merge.

Tooling

mergekit (Arcee AI): toolkit chuẩn de-facto. YAML config, support tất cả methods. `pip install mergekit`.

Use cases

Multi-skill: merge math-FT + code-FT + chat-FT. **Continual learning**: merge base + new-domain FT. **Open LLM Leaderboard**: phần lớn top models 2024 là merges.

Quantization & serving — training xong, deploy thế nào?

Quantization (training-free):

- **NF4** (QLoRA): 4-bit normal-aware — default FT.
- **GPTQ**: post-training 4-bit + calibration set.
- **AWQ**: activation-aware, tốt hơn GPTQ ở 4-bit.
- **GGUF**: format llama.cpp (Q4_K_M…).

Serving:

- **vLLM**: continuous batching, PagedAttention.
- **Speculative decoding**: 2–3× throughput.
- **KV cache quant**: 8/4-bit cho long-context.

Advanced training & mid-training:

- **Long-context** (mid-train): RoPE scaling — NTK, YaRN, LongRoPE → 4K → 128K+.
- **MoE fine-tune**: FT subset experts.
- **Multi-token prediction**: predict *n* forward.
- **Curriculum learning**: easy-to-hard (Phi-4).
- **Pruning**: SparseGPT, Wanda — ít phổ biến.

Lưu ý: Đừng dùng tất cả cùng lúc. Thêm từng cái, đo, giữ cái work.

Decision tree — chọn training strategy theo constraint

1. Có **preference pairs**?
   - ▷ +SFT model → **DPO / SimPO**
   - ▷ Không SFT → **ORPO** (1-stage)
   - ▷ Chỉ **+1**/**−1** → **KTO**
2. Có **instruction data**? → **SFT (LoRA/QLoRA)**, rồi thu preference → DPO.
3. Chỉ **raw domain text**?
   - ▷ Xa base (medical/legal/code) → **DAPT** → **TAPT** → **SFT** → **DPO**.
   - ▷ Gần base → skip DAPT, SFT với synthetic data.
4. Math/code/**ground truth**? → sau DPO thêm **GRPO + RLVR** (Tulu 3).
5. Nhiều **task-specific FT models** cần combine? → **Model merging** (TIES/DARE/SLERP).
6. Cần **deploy efficient**? → Quantize (NF4/AWQ/GGUF) + vLLM/llama.cpp + speculative decoding.

Lưu ý: Reality: hầu hết enterprise teams chạy **SFT + DPO** là đủ. Phần còn lại là "nice to have".

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. DPO vẫn là go-to 2025-2026 — nhưng phải **iterative** (Llama 3 / Tulu 3), không one-shot
2. Watch failure modes: likelihood displacement, length hacking. `rewards/chosen` đảo chiều = stop
3. ORPO khi base→aligned 1 stage; SimPO khi length bias là vấn đề; KTO khi chỉ có +1/−1
4. RL trở lại với GRPO + RLVR cho reasoning — *không* reward model. Stack: SFT → DPO → RLVR
5. VN-first models dừng ở SFT — Lab 22 Bonus B là cơ hội publish DPO-aligned VN model

Tiếp theo & Bài tập

Ngày 23: LangGraph & Agentic Orchestration

"Model đã aligned. Tiếp theo: orchestrate complex workflows với LangGraph stateful machines."

- Hoàn thành Lab 22: DPO alignment + deploy aligned model
- Đọc: LangGraph documentation — State, Nodes, Edges

Hỏi & Đáp

DPO hay ORPO — bạn sẽ chọn method nào cho project của mình và tại sao?



Cảm ơn!

AICB-P2T3 · Ngày 22 · DPO, ORPO & Alignment — Từ SFT đến Preference Learning

github.com/VinUni-AI20k

Liên hệ: instructor@vinuni.edu.vn